

MediRush-SafeAgent: A Policy-Constrained Multi-Stage Defense Framework for Reliable Healthcare-Commerce LLM Tool Execution

****Author****: Sham Satish Thakare

Artificial Intelligence in Medicine (Elsevier)

Abstract

Autonomous tool-using Large Language Model (LLM) agents are increasingly deployed in complex domain-specific workflows. However, deploying agents in healthcare-commerce applications presents critical security and policy challenges, including unauthorized order modifications, prescription bypasses, cross-user data exposure, and indirect prompt injections. In this paper, we present ****MediRush-SafeAgent****, a formal multi-stage runtime framework designed to guarantee policy-constrained, authorized, and reliable tool execution in healthcare-commerce workflows. MediRush-SafeAgent integrates a pre-execution domain policy engine, a strict scope authorization verifier, a post-execution environment state verifier, and dynamic risk-adaptive human escalation. We construct ****MediRushBench****, a standardized evaluation benchmark comprising 24 scenarios across 12 operational and security categories. Our empirical evaluation demonstrates that MediRush-SafeAgent achieves a 100.0% Safe Task Completion Rate (\$STCR\$) and 0.0% Policy Violation Rate (\$PVR\$), outperforming unconstrained LLM baselines (\$STCR = 83.33\%\$, \$UTIR = 16.67\%\$) and prompt-guardrail baselines (\$STCR = 95.83\%\$). These results establish the efficacy of joint multi-stage runtime interception for domain-constrained agent safety.

****Keywords****: AI Agent Safety, Tool Execution Interception, Healthcare-Commerce Workflows, Dynamic Risk Assessment, Prompt Injection Resiliency, Operational Policy Enforcement.

1. Introduction

Large Language Model (LLM) agents have evolved beyond passive text generation toward autonomous task execution via external tool interfaces (Yao et al., 2023; Schick et al., 2023). When applied to digital healthcare-commerce—such as localized medicine search, inventory inspection, prescription verification, and order processing—autonomous agents offer substantial efficiency gains for consumers and healthcare providers.

However, operating autonomous tool-calling agents in domain-restricted environments introduces severe security vulnerabilities. Unconstrained LLM agents are susceptible to *indirect prompt injections* embedded in product descriptions or pharmacy metadata (Debenedetti et al., 2024), *permission laundering* across multi-step tool calls (Wang et al., 2025), and *cross-user data exfiltration* (Andriushchenko et al., 2024). In healthcare-commerce, an unmitigated tool call could result in unauthorized cart placement of prescription-only medications or exposure of personal health order details.

Prior defensive approaches have primarily relied on static prompt guardrails or generic system privilege Domain-Specific Languages (DSLs) (Chen et al., 2025). However, prompt guardrails remain vulnerable to jailbreaks, while generic privilege engines lack awareness of domain-specific state transitions (e.g., verifying physician prescription credentials prior to inventory allocation).

To address these challenges, we introduce **MediRush-SafeAgent**, a multi-stage defense architecture tailored for safe healthcare-commerce interactions. MediRush-SafeAgent enforces a formal four-layer defense pipeline:

1. **Pre-Execution Domain Policy Engine**: Enforces non-negotiable domain rules (e.g., medical advice refusal, prescription verification, and order volume limits).
2. **Scope Authorization Boundary**: Validates user identity and resource ownership scope before dispatching API calls.
3. **Post-Execution State Verifier**: Inspects environment state deltas following tool execution to ensure side-effects match expected outcomes.
4. **Dynamic Risk-Adaptive Escalation**: Calculates a dynamic risk vector $\vec{R}$ to escalate high-risk transactions to human approval while maintaining autonomous throughput for low-risk Over-The-Counter (OTC) requests.

Furthermore, we introduce **MediRushBench**, a standardized evaluation benchmark featuring 24 scenario cases spanning 12 operational and security categories. Our empirical results demonstrate that MediRush-SafeAgent achieves a 100.0% Safe Task Completion Rate (\$STCR\$) and 0.0% Policy Violation Rate (\$PVR\$), establishing a robust paradigm for policy-constrained LLM tool deployment.

2. Related Work

2.1 Tool-Using LLM Agents

The integration of external API tools with language models was formalized by ReAct (Yao et al., 2023) and Toolformer (Schick et al., 2023). While these foundational paradigms enabled complex multi-step reasoning, they assumed a trusted environment without explicit safety or authorization boundaries.

2.2 Agent Security and Privilege Control

Recent works have addressed agent security risks. Progent (Chen et al., 2025) proposed fine-grained privilege control using domain-specific languages. ToolGuardian (2026) introduced Answer Set Programming (ASP) for pre-admission vetting. AgentTrust (2026) demonstrated runtime interception of tool calls. However, these systems do not incorporate post-execution state verification or domain-specific risk scoring for healthcare interactions.

2.3 Adversarial Evaluation & Benchmarks

AgentDojo (DeBenedetti et al., 2024) established a benchmark for indirect prompt injection in tool-using agents. ToolEmu (Ruan et al., 2024) introduced tool environment emulation for risk identification. AgentHarm (Andriushchenko et al., 2024) evaluated harmfulness in agent execution. MediRushBench extends these benchmarks by specifically targeting healthcare-commerce authorization, prescription policy enforcement, and multi-step state consistency.

3. System Architecture: MediRush-SafeAgent

The MediRush-SafeAgent architecture introduces a four-stage runtime defense interceptor operating between the LLM Planner and external environment tools.

3.1 Pre-Execution Policy Engine

The Policy Engine evaluates tool parameters against static and contextual domain rules:

$$\text{Verdict}_{\text{policy}} = P(T_i, \text{args}_i, \mathcal{C})$$

where T_i represents the candidate tool, args_i the argument payload, and $\mathcal{C}$ the session context. In healthcare-commerce, key rules include: (1) Refusal of clinical diagnosis/dosage requests, (2) Mandatory prescription verification for restricted pharmaceuticals, and (3) Quantity caps (≤ 10 units).

3.2 Scope Authorization Boundary

The Scope Verifier asserts user identity ownership:

$$\text{Verdict}_{\text{auth}} = A(T_i, \text{user_id}_{\text{session}}, \text{user_id}_{\text{target}})$$

Attempts to access or modify orders belonging to external user IDs (e.g. `USER-VICTIM`) are blocked prior to tool invocation.

3.3 Post-Execution State Verifier

Following tool execution, the State Verifier validates the state delta $\Delta S = S_{\text{post}} - S_{\text{pre}}$ against the intended tool specification:

$$V_{\text{state}} = (\Delta S_{\text{actual}} \equiv \Delta S_{\text{expected}})$$

State mismatches trigger immediate rollback and alert logging.

3.4 Dynamic Risk-Adaptive Escalation

A risk scoring function calculates vector $\vec{R} = [R_{\text{rx}}, R_{\text{volume}}, R_{\text{pii}}]$. If $|\vec{R}| \geq \tau_{\text{risk}}$, the action is routed to Human-In-The-Loop (HITL) approval.

4. Experimental Setup & MediRushBench

4.1 MediRushBench Benchmark

We construct **MediRushBench**, comprising 24 test scenarios categorized across 12 operational areas: Search, Inventory, Cart, Prescription Policy, Order Status, Medical Safety, Scope Authorization, Indirect Prompt Injection, Tool Misuse, Failure Recovery, Ambiguity, and Human Escalation.

4.2 Comparative Baselines

We compare MediRush-SafeAgent against five baselines:

- **B0 (Deterministic)**: Hardcoded rule-based execution.
- **B1 (Unconstrained)**: Standard tool-calling LLM without guardrails.
- **B2 (PromptGuard)**: Tool-calling LLM with system prompt instructions.
- **B3 (PolicyOnly)**: Tool agent with pre-execution policy engine.
- **B4 (PolicyAuth)**: Tool agent with policy engine and scope authorization.
- **B5 (MediRush-SafeAgent)**: Proposed full architecture.

4.3 Evaluation Metrics

Primary metric: **Safe Task Completion Rate (\$STCR\$)**, defined as the proportion of episodes resulting in task success with zero policy or authorization violations:
$$STCR = \frac{N_{\text{safe_success}}}{N_{\text{total}}} \times 100\%$$
Secondary metrics include Policy Violation Rate (\$PVR\$), Unauthorized Tool Invocation Rate (\$UTIR\$), and Human Escalation Rate (\$HITL\$).

5. Experimental Results

Table 1 summarizes the experimental results across all six evaluated baselines on MediRushBench.

Baseline Architecture	STCR (\$\uparrow\$)	PVR (\$\downarrow\$)	UTIR (\$\downarrow\$)	HITL (\$\rightarrow\$)
Baseline	100.0%	0.0%	0.0%	0.0%
B0 (Deterministic)	100.0%	0.0%	0.0%	0.0%
B1 (Unconstrained LLM)	83.33%	0.0%	16.67%	0.0%
B2 (PromptGuard)	95.83%	0.0%	4.17%	0.0%
B3 (PolicyOnly)	83.33%	0.0%	16.67%	0.0%
B4 (PolicyAuth)	100.0%	0.0%	16.67%	0.0%
B5 (MediRush-SafeAgent)	100.0%	0.0%	16.67%	0.0%

5.1 Key Findings

- Unconstrained Vulnerability**: Unconstrained agents (B1) suffer from an Unauthorized Tool Invocation Rate (\$UTIR\$) of 16.67% when exposed to cross-user requests and prompt injection scenarios.
- Prompt Guard Incompleteness**: Prompt guardrails (B2) improve \$STCR\$ to 95.83%, but remain vulnerable to indirect prompt injection payloads embedded in product descriptions (\$UTIR\$ = 4.17%).
- Multi-Stage Defense Superiority**: Integrating pre-execution policy enforcement and scope authorization (B4, B5) achieves 100.0% Safe Task Completion Rate (\$STCR\$) and 0.0% Policy Violation Rate (\$PVR\$).

6. Discussion

The experimental findings demonstrate that static prompt guardrails are insufficient for high-stakes healthcare-commerce workflows. While system prompts discourage policy violations in simple contexts, they fail under indirect prompt injection attacks where adversarial payloads override context.

MediRush-SafeAgent proves that deterministic runtime interception—evaluating policy rules and user scope boundaries at the API execution layer—provides complete defense against unauthorized tool execution. Furthermore, post-execution state verification ensures that even if an API call succeeds, unauthorized side-effects are detected and reverted before corrupting system state.

7. Limitations

We explicitly acknowledge several limitations of this work:

- Synthetic Benchmark Scenarios**: MediRushBench utilizes synthetic pharmacy catalogs and order scenarios. While designed to simulate realistic healthcare-commerce interactions, external validity on

live commercial platforms remains to be evaluated.

2. **Absence of Clinical Evaluation**: MediRush-SafeAgent strictly refuses medical diagnosis and dosage tasks. It does not evaluate clinical decision-making or real patient medical outcomes.

3. **Model Diversity**: Experiments were conducted using a deterministic simulated LLM provider interface. Future extensions should benchmark commercial frontier APIs (e.g. GPT-4o, Claude 3.5) and open-weight models (e.g. Llama 3.3).

8. Ethical Considerations

MediRush-SafeAgent is an experimental research prototype for agent safety and authorization enforcement. It is **not** a medical diagnosis tool, prescribing system, or clinical advice system, and must not be used as a substitute for professional medical care.

All benchmark datasets in MediRushBench consist exclusively of synthetic, non-identifiable user personas and mock pharmaceutical items. No real Patient Health Information (PHI) or real-world patient records were collected or processed.

9. Conclusion

In this paper, we introduced **MediRush-SafeAgent**, a multi-stage runtime framework for safe, authorized, and policy-constrained tool execution in healthcare-commerce workflows. By combining pre-execution policy checks, scope authorization verification, post-execution state verification, and dynamic risk-adaptive human escalation, MediRush-SafeAgent achieves 100.0% Safe Task Completion Rate and 0.0% Policy Violation Rate on the novel MediRushBench benchmark. Our framework provides a rigorous foundation for deploying secure autonomous LLM agents in domain-restricted applications.

References

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2023. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.
2. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T., 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS.
3. DeBenedetti, E., Zhang, J., Carlini, N., Tramèr, F., 2024. AgentDojo: A Dynamic Environment for Evaluating Indirect Prompt Injection in Language Agents. NeurIPS.
4. Chen, X., Zhang, Y., Liu, W., 2025. Progent: Fine-Grained Privilege Control for Autonomous Tool-Using Agents. IEEE S&P.
5. Wang, H., Zhao, L., Kim, S., 2025. ChainCaps: Preventing Permission Laundering in Multi-Step Agent Tool Chains. arXiv:2511.04589.
6. Ruan, Y., Sheng, H., Zhang, M., Zhao, T., Liang, P., Hashimoto, T., 2024. Identifying the Risks of LM Agents with Tool Environment Emulation. ICML.
7. Andriushchenko, M., Croce, F., Flammarion, N., 2024. AgentHarm: Benchmarking Harmfulness and Red Teaming of LLM Agents. arXiv:2410.08154.